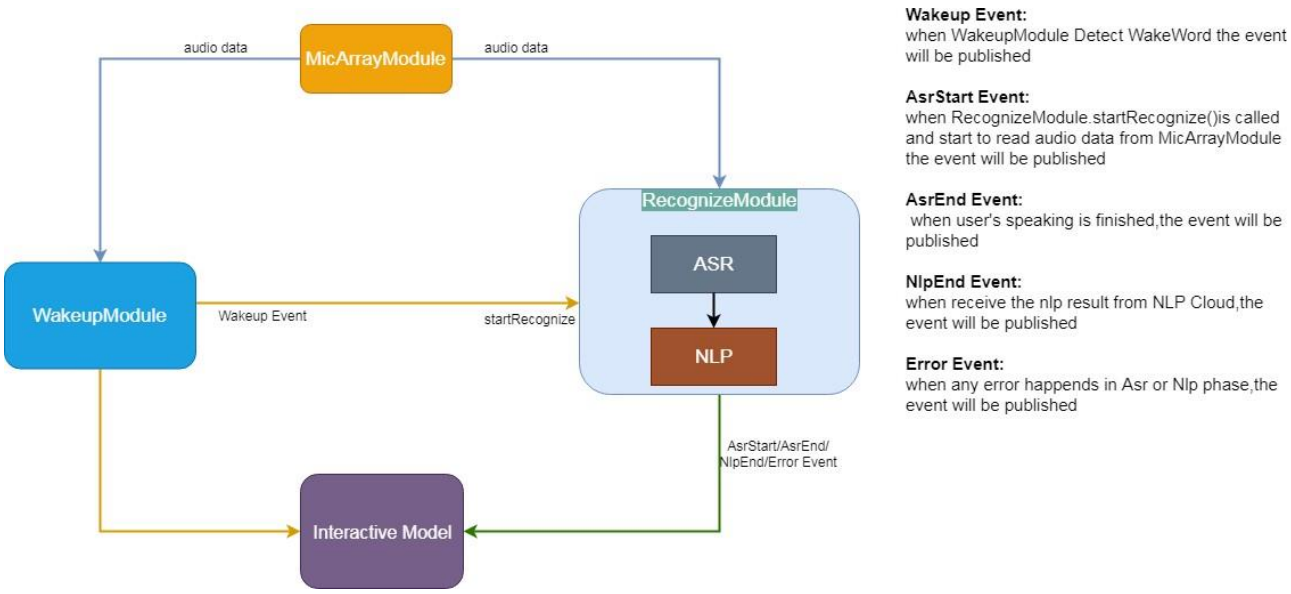
Development guide of speech service for Alpha Mini

I. Overview

Speech service is a basic service application in Alpha Mini robot system, which, as a system input application, can receive the user's voice input and output the corresponding voice command. Alpha Mini robots are officially only available in Chinese, English and Korean in speech recognition, and developers may want Alpha Mini to support more languages, so they need access to new voice service providers (such as Alexa, Google Assistant, etc.) if they want the experience of voice interaction to be as effective as the official version of Alpha Mini. Developers need to do a lot of development work. To this end, we have developed a development framework, The Speech Framework, for fast access. The framework integrates the microphone array module, wake-up module, recognition module, TTS module, and interactive model, and enables automatic switching at all stages. It enables the switching of the microphone array, the reading of recording data, the positioning of the sound source, and the realization of the letter, realize sepsis, automatically turn off the microphone array when you hibernate, and re-open the function when activated. Developers only need to follow the interface we define to achieve wake-up, recognition, TTS modules, you can achieve the complete and Alpha Mini robot built-in voice module suffly interactive performance.

The basic framework is as below:



II. The core module description

Mic Array Module: Read the original audio data from 4-Mic and output 16K HZ, 16bit, single-channel audio data after processing with the microphone array front-end algorithm.

Wakeup Module : Enables wake-up to monitor if a voice wake-up occurs by constantly reading microphone data input from Mic Array Module to the wake-up engine, and triggers Recognize Module into the recognition phase by publishing a wake-up event when a voice wake-up event occurs.

Recognize Module: Including ASR, NLP, TTS process, developers can integrate Amazon AVS and Tencent Tinker, such as the end-to-end full voice services, or separately access ASR API, NLP API, TTS API to achieve voice interaction.

Interactive Model: When a robot is woken up by a voice, when the robot is in an identification state, when the robot is in the waiting stage for recognition results, the robot has different performance, in order to ensure the consistency of interaction, we have implemented a standard interaction model, developers do not need to develop additional. Wake-up: broadcast wake-up response audio, the eye play smile animation, turn the head sound source positioning, speech recognition stage: eye play blink animation, NLP recognition stage: eye play left and right rotation animation;

Note: Developers can customize wake-up responses by passing in the id of the audio resource and allow multiple incomings; The timing of the turnsource positioning can be customized to support wake-up or at the end of speech recognition.

III. Coding implementation

3.1 How to gain access to the Mic Array

1. Add label

In `AndroidManifest.xml` application label, add the following code to request access to the microphone array

```
<meta-data android:name="ubt-master-app"
android:value="third_part_speechservice"/>
```

2, After the first installation of the apk, need to restart the robot

The robot checks the installed apk for `third_part_speechservice` mark each time it is turned on, and if so, the disable system's built-in access microphone array of the app Speech Service, the microphone array's access to the apk. Once the system's built-in Speech Service is used, users will not be able to access the built-in voice service.

3.2 Give out microphone array access

1. Uninstall the app or remove the label of meta-data

2. Restart the robot

- After restarting the robot, the system's built-in Speech Service will regain access to the microphone array, and users can continue to use the built-in voice service.

3.3 Input SDK

1、Download demo speechFrameworkdemo

Including sdk as below：

```
behavior-lib-versionCode.aar
impl-wakeup-dingdang-versionCode.aar mini-
outer-sdk-lite-versionCode.jar sal-speech-
versionCode.jar
speechFramework-oversea-release-versionCode.aar tencent-
vad-versionCode.aar
utlilib-versionCode.aar
weinalib-versionCode.aar
```

2、sdk input:

Configure in the build.gradle file for app module as follows:

```
android {
     packagingOptions {
     pickFirst 'lib/arm64-v8a/libtvad.so'
     pickFirst 'lib/armeabi-v7a/libtvad.so'
   }
   compileOptions { sourceCompatibility 1.8
     targetCompatibility 1.8
   }
   repositories{flatDir{dirs 'libs'}}
}


dependencies {
   implementation fileTree(dir: 'libs', include: ['*.jar']) implementation
   "org.jetbrains.kotlin:kotlin-stdlib-jdk7:1.3.21" implementation(name:
   'speechFramework-oversea-release', ext: 'aar') implementation(name: 'behavior-
   lib-1.0.0', ext: 'aar') implementation(name: 'impl-wakeup-dingdang-1.0.0', ext:
   'aar') implementation(name: 'sal-speech-1.0.0', ext: 'aar') implementation(name:
   'tencent-vad-1.0.0', ext: 'aar') implementation(name: 'utillib-1.2.8', ext:
   'aar') implementation(name: 'weinalib-4.0.0', ext: 'aar')
}
```

3、Initialization:

Add the following code to the method of the `Application` subclass's `onCreate()`:

```
     //start a MasterService, and init modules in DemoSpeech.
     startService(new Intent(this, DemoMasterService.class));

     //declare SpeechSettings
     ServiceModules.declare(SpeechSettings.class,
          (aClass, moduleCreatedNotifier) ->
moduleCreatedNotifier.notifyModuleCreated(
               DemoSpeech.INSTANCE.createSpeechSettings()));

     //declare SpeechService
     ServiceModules.declare(SpeechService.class,
          (aClass, moduleCreatedNotifier) -> ThreadPool.runOnNonUIThread(()
-> {
            while (DemoSpeech.INSTANCE.getSpeechService() == null)
               { SystemClock.sleep(5);
            }
            Log.d("Speech", "Speech Service create ok..");

moduleCreatedNotifier.notifyModuleCreated(DemoSpeech.INSTANCE.getSpeechSer vice());
         }));
```

DemoMaster Service is a Master service that is required to publish (publishCarefully) events in DemoSpeech.
SpeechSettings are configurable parameters for voice services, such as 'recognition_language' to indicate the recognition language; 'speaking_language' represents the tts language; wake-up, recognition, tts module can listen to parameter changes in one way:

```
ServiceModules.getModuleCreator(SpeechSettings.class).createModule(SpeechS ettings.class,
new ModuleCreatedNotifier<SpeechSettings>() {
       @Override public void notifyModuleCreated(SpeechSettings
speechSettings) {
          speechSettings.registerListener(new SpeechConfigurationListener()
{
            @Override public void onConfigurationChanged(SpeechConfiguration
speechConfiguration) {
               //listening to speech configuration
            }
          });
       }
     });
```

SpeechService inherited the four modules of wake-up, compositing, speech recognition, semantic understanding:

```
public interface SpeechService extends WakeUpDetector, Synthesizer, Recognizer,
Understander {
}
```

The initialization and construction of each module of the voice service is completed within DemoSpeech. It provides an interface to createSpeech Service (), when
After The Speech Service initialization is successful, register with The ServiceModulus:

```
    val wakeUpDetector = WeiNaWakeUpDetector(WeiNaRecorder(false)) recognizer
    = DemoRecognizer(asrRecorder)
    synthesizer = DemoSynthesizer()
    understander = DemoUnderstander()
    speechServiceStub =
CompositeSpeechService.Builder().setRecognizer(recognizer)
                            .setSynthesizer(synthesizer)
                            .setUnderstander(understander)
                            .setWakeUpDetector(wakeUpDetector)
                            .build()
```

3.3 Wakeup Module realization

You can use the built-in wake-up module WeiNaWakeUpDetector directly, and the supported wake-up word is "Hi, mini", which allows you to negotiate business and provide training data if you need to define a new wake word.

If a third-party company wants to use its own wake-up technology, it needs to implement the `AbstractWakeUpDetector` interface

How to implement `AbstractWakeUpDetector` can be referred within The `WeiNaWakeUpDetector`, which obtains recording data from the microphone array:

```
    audioRecorder.start();
    audioRecorder.registerRecordListener(new AudioRecordListener() { public void
      onRecord(byte[] audio, int size) {
        if (!DingdangWakeUpDetector.this.mThreadNameSet)
          { DingdangWakeUpDetector.this.mThreadNameSet = true;
          Thread.currentThread().setName("wake_up_audio_append");
        }

        //asrData: pcm, 16000 sampleRate, 8bit
        //Receive the recording data of microphone output in line here
        //size = 2048
        //todo ...
      }
    }, Executors.newSingleThreadExecutor(), "all");
```

3.4 Synthesizer Module realization

When developers in RecognizeModule are connected to a complete voice service platform such as Amazon AVS or Tencent Tinker, the results of NLP recognition are synthesized directly in the cloud and then returned to the robot side. This situation may not require the implementation of the Temper Module, but other modules of the robot need to be implemented if they need to implement tts broadcast, such as in an app to implement the function of face recognition, the need to identify the human name broadcast, In another app to implement the ability to power up the weather forecast, all need to use the tts function, then need to implement TTSModule, in other apps developers can provide through the `VoicePool.playTTS ()` Method to pass the tts request to The Torch Module.

> a、Implement Synthesizer interface

```
public abstract class AbstractSynthesizer implements Synthesizer { protected abstract

  void startSynthesizing(SynthesisOption option); protected abstract void

  stopSynthesizing();

  List<SpeakingVoice> getSpeakingVoiceList();
}
```

`startSynthesizing()`: Third-party implementation sound synthesis `stopsynthesizing ()`: Third-party implementation stop sound synthesis, can release resources `getSpeakering VoiceList ()`: third-party implementation, supported pronunciation list. `SynthesisOption` is a parameter used for sound synthesis, with the following important properties:

```
private String inputText;//要合成的文本private
String speakingVoiceId;//发音人id
```

```
private float speakingSpeed;//发音人语速private
int speakingVolume;//发音人音量private String
languageCode;//语言, 如zh_CN
```

3.5 Recognize Module realization

> a、Implement AbstractRecognizer interface

```
public abstract class AbstractRecognizer implements Recognizer { protected abstract
  void startRecognizing(RecognitionOption option);

  protected abstract void stopRecognizing();
  //...
}
```

In this Module can access the complete voice services such as Amazon AVS, Tencent dingdang, and also ASR, NLP, TTS API.

- `startRecognizing()`:implement asr from third-party, you can use `TencentVad Recorder` to obtain the microphone column recording data, when need to return to identify the progress, you can use `reportRecognizingProgress(RecognitionProgress progress)`, if fail and end,then use `resolveRecognizing(RecognitionResult done)`; if fail to recognize, use `rejectRecognizing(RecognitionException e)`。
- `stopRecognizing()`:If the asr has been started, the user wakes up the robot at this time and needs to stop the asr, the framework will call this method, and then the framework restarts the asr process. There's some cleanup in `stopRecognize ()`.
- `RecognitionOption`: `Recognizer` specifications

```
  public static final int MODE_SINGLE = 1;// single recognize
  public static final int MODE_CONTINUOUS = 2; //contimuous recognize private int
  mode;// recognize mode
  private boolean withUnderstanding; //after recognized, return
Understander result
  private int distanceRange;
  private long timeoutMillis;
  private JsonObjectString extension;
  private UnderstandingOption understandingOption;//understander param
```

3.6 Use **TencentVadRecorder** to access **asr** recording data

> a, the framework will start the microphone column to start recording when the application starts, you can use `TencentVadRecorder` to obtain microphone recording data. For example:

```
public class DemoRecognizer extends AbstractRecognizer { private
  final TencentVadRecorder recorder;
```

```
    public DemoRecognizer(TencentVadRecorder recorder)
      { this.recorder = recorder;
      //after recorder.start() called, you could accept asr data.
      recorder.registerRecordListener(new AudioRecordListener() {
        @Override public void onRecord(byte[] asrData, int length) {
          //asrData: pcm, 16000 sampleRate, 8bit
          //Receive the recording data of microphone output in line here
        }
      }, null, null);
    }

    @Override protected void startRecognizing(RecognitionOption
  recognitionOption) {
      //after accept asr request, you should start recording
      recorder.start();
    }

    @Override protected void stopRecognizing()
      { recorder.stop();
    }
  }
```

b、 TencentVadRecorder support vad test , after recorder.start() ,public void registerStateListener(AudioRecordStateListener audioRecordStateListener)Listening status , STATE_SPEAK_BEGAN detect people start to speak, STATE_SPEAKING detect people are speaking , STATE_SPEAK_END detect speak end, STATE_NO_SPEAK detect no people speak; when STATE_SPEAK_BEGAN, will pass the asr data through AudioRecordListener to Recognizer.

```
    public static int STATE_IDLE = 0; public
    static int STATE_RECORDING = 1; public static
    int STATE_ERROR = 2;
    public static int STATE_SPEAK_BEGAN = 3;// vad speak begin public
    static int STATE_SPEAKING = 4;// vad speaking public static int
    STATE_NO_SPEAK = 5;//vad no speaking public static int
    STATE_SPEAK_END = 6;//vad speak end
```

c、enableBeamforming(), disableBeamforming()Controlable microphone beams.

3.7 Understander Module realization

a、Implement AbstractUnderstander Interface

```
  public abstract class AbstractUnderstander implements Understander { protected
    abstract void understand(final UnderstandingOption option,
        final Deferred<UnderstandingResult, UnderstandingException> deferred);
  }
```

*understand(): third party implement nlp, can receive the results returned by Recognizer to fill in
UnderstandingOption When the nlp process changes, report NLP results through Deferred<UnderstandingResult, UnderstandingException>.

*UnderstandingOption: Understander specification

```
    private String inputText; //asr result. private
    String version;
    private String sessionId;
    private JsonObjectString parameters; //nlp request parameters private
    JsonArrayString contextList; //context info
    private long timeoutMillis;
```

> b、b, like AVS voice engine, will bring back NLP results after Recognizer recognition; So the implementation of the Understander understand() approach may only require reassembling the identification results into UnderstandingResult.